Chapitre 7 : PROGRAMMATION ORIENTÉE OBJET EN JAVA

Par Robert DIASSÉ

Introduction générale

La programmation orientée objet (POO) est une approche de programmation qui consiste à organiser un programme autour d'objets représentant des entités du monde réel ou du problème à résoudre.

Un objet regroupe :

- des **données**, appelées attributs ;
- des **actions**, appelées méthodes.

L'objectif de la POO est de produire des programmes :

- mieux structurés,
- plus faciles à comprendre,
- plus simples à maintenir et à faire évoluer.

---

1. Notion de classe

Une **classe** est un modèle, un plan de fabrication qui décrit :

- les données qu'un objet possédera ;
- les actions qu'un objet pourra effectuer.

Une classe ne représente pas encore un objet réel.
Elle définit uniquement ce que sera un objet lorsqu'il sera créé.

---

2. Organisation des fichiers Java

En Java, la règle est la suivante :

- Une classe publique doit être déclarée dans un fichier portant **exactement le même nom**.
- Un fichier Java contient en général **une seule classe publique**.

Exemple :

- `Etudiant.java` contient la classe `Etudiant`
- `TestEtudiant.java` contient la classe `TestEtudiant`

---

3. Déclaration complète d'une classe

Fichier : `Etudiant.java`

```
public class Etudiant {

    private String nom;
    private int age;
    private double moyenne;

    public Etudiant(String nom, int age, double moyenne) {
        this.nom = nom;
        this.age = age;
        this.moyenne = moyenne;
    }

    public void afficher() {
        System.out.println("Nom : " + nom);
        System.out.println("Age : " + age);
        System.out.println("Moyenne : " + moyenne);
    }
}
```

Explication

- `public class Etudiant` définit une classe accessible depuis d'autres fichiers.
- Les attributs sont déclarés avec `private` pour empêcher un accès direct.
- Le constructeur initialise l'état de l'objet lors de sa création.
- La méthode `afficher()` permet d'afficher les informations de l'objet.

---

4. Création et utilisation d'objets

Fichier : `TestEtudiant.java`

```
public class TestEtudiant {
    public static void main(String[] args) {

        Etudiant e1 = new Etudiant("Aminata", 20, 15.5);
        Etudiant e2 = new Etudiant("Moussa", 22, 13.0);

        e1.afficher();
        e2.afficher();
```

```
    }
}
```

Résultat attendu

```
Nom : Aminata
Age : 20
Moyenne : 15.5
Nom : Moussa
Age : 22
Moyenne : 13.0
```

Explication

- Le mot-clé `new` permet de créer un objet réel à partir de la classe.
- Chaque objet possède ses propres valeurs.
- Les méthodes sont appelées à partir de l'objet.

---

5. Le constructeur

Le **constructeur** est une méthode spéciale :

- portant le même nom que la classe ;
- appelée automatiquement lors de la création d'un objet ;
- utilisée pour initialiser les attributs.

```java
public Etudiant(String nom, int age, double moyenne) {
    this.nom = nom;
    this.age = age;
    this.moyenne = moyenne;
}
```

---

6. Le mot-clé `this`

Le mot-clé `this` représente l'objet courant.

Il permet de distinguer :

- l'attribut de la classe ;
- le paramètre reçu par le constructeur ou la méthode.

Sans `this`, Java ne peut pas savoir à quel élément on fait référence.

---

7. Encapsulation

L'encapsulation consiste à protéger les données internes d'un objet.

Les attributs sont déclarés en `private` afin d'empêcher toute modification directe depuis l'extérieur.

```
private double moyenne;
```

Cette protection évite les incohérences et les erreurs logiques.

---

8. Getters et setters

Les **getters** permettent de lire une valeur.
Les **setters** permettent de modifier une valeur de manière contrôlée.

```
public double getMoyenne() {
    return moyenne;
}

public void setMoyenne(double moyenne) {
    if (moyenne >= 0 && moyenne <= 20) {
        this.moyenne = moyenne;
    }
}
```

Grâce aux setters, on peut vérifier la validité des données avant modification.

---

9. Héritage

L'héritage permet de créer une nouvelle classe à partir d'une classe existante.

La nouvelle classe hérite des attributs et méthodes de la classe parent.

Fichier : `Personne.java`

```
public class Personne {
    protected String nom;

    public Personne(String nom) {
        this.nom = nom;
    }

    public void afficher() {
        System.out.println("Nom : " + nom);
    }
}
```

Fichier : `Etudiant.java`

```
public class Etudiant extends Personne {
    private String matricule;

    public Etudiant(String nom, String matricule) {
        super(nom);
        this.matricule = matricule;
    }

    @Override
    public void afficher() {
        super.afficher();
        System.out.println("Matricule : " + matricule);
    }
}
```

---

10. Polymorphisme

Le polymorphisme permet de manipuler un objet à travers une référence de son type parent.

```
Personne p = new Etudiant("Ali", "ISI2025");
p.afficher();
```

Java appelle automatiquement la version de la méthode correspondant au type réel de l'objet.

---

11. Classes abstraites

Une **classe abstraite** représente un concept incomplet.

Caractéristiques :

- elle ne peut pas être instanciée ;
- elle peut contenir des méthodes abstraites ;
- elle sert de base à d'autres classes.

Fichier : `Animal.java`

```
public abstract class Animal {

    protected String nom;

    public Animal(String nom) {
        this.nom = nom;
    }
```

```
    public abstract void crier();

    public void dormir() {
        System.out.println(nom + " dort");
    }
}
```

Fichier : `Chien.java`

```
public class Chien extends Animal {

    public Chien(String nom) {
        super(nom);
    }

    @Override
    public void crier() {
        System.out.println(nom + " aboie");
    }
}
```

---

12. Interfaces

Une interface définit un contrat que les classes doivent respecter.

Une interface :

- ne contient que des méthodes abstraites ;
- impose un comportement commun ;
- permet une implémentation multiple.

Fichier : `Payable.java`

```
public interface Payable {
    void payer(double montant);
}
```

Fichier : `Employe.java`

```
public class Employe implements Payable {

    private String nom;

    public Employe(String nom) {
        this.nom = nom;
```

```
    }

    @Override
    public void payer(double montant) {
        System.out.println(nom + " reçoit " + montant + " FCFA");
    }
}
```

---

13. Différence entre classe abstraite et interface

- Une classe abstraite représente un concept partiellement défini.
- Une interface représente un engagement à fournir un comportement.

Une classe ne peut hériter que d'une seule classe abstraite, mais peut implémenter plusieurs interfaces.

Conclusion générale

La programmation orientée objet permet :

- une meilleure organisation du code ;
- une modélisation fidèle du problème ;
- une maintenance facilitée ;
- une évolution progressive des applications.

Les principes fondamentaux sont :

- l'encapsulation ;
- l'héritage ;
- le polymorphisme ;
- l'abstraction.

Parfait.
Voici **5 exercices POO Java intéressants, progressifs et entièrement corrigés indiqués**, conçus pour **donner une vraie vision de ce que permet la POO**, tout en restant **accessibles** et **pédagogiques**.
Le style est volontairement **sobre**, **structuré**, et prêt à être **intégré directement dans ton cours**.

EXERCICES POO JAVA – SÉRIE D'APPLICATION COMPLÈTE (AVEC CORRECTIONS)

Tester absolument et améliorer ces exercices pour entiérement comprendre

Exercice 1 – Gestion d'un étudiant (classe simple + encapsulation)

Énoncé

Créer une classe `Etudiant` qui contient :

- le nom,
- l'âge,
- la moyenne.

Les données doivent être protégées.
Prévoir un constructeur, des getters/setters, et une méthode permettant d'indiquer si l'étudiant est admis (moyenne ≥ 10).

Correction

Fichier : `Etudiant.java`

```
public class Etudiant {

    private String nom;
    private int age;
    private double moyenne;

    public Etudiant(String nom, int age, double moyenne) {
        this.nom = nom;
        this.age = age;
        this.moyenne = moyenne;
    }

    public boolean estAdmis() {
        return moyenne >= 10;
    }

    public void afficher() {
        System.out.println("Nom : " + nom);
        System.out.println("Age : " + age);
        System.out.println("Moyenne : " + moyenne);
        System.out.println("Résultat : " + (estAdmis() ? "Admis" : "Ajourné"));
    }
}
```

Fichier : `TestEtudiant.java`

```
public class TestEtudiant {
    public static void main(String[] args) {
        Etudiant e = new Etudiant("Fatou", 21, 12.5);
        e.afficher();
    }
}
```

Résultat attendu

```
Nom : Fatou
Age : 21
Moyenne : 12.5
Résultat : Admis
```

Exercice 2 – Compte bancaire (comportement + contrôle logique)

Énoncé

Créer une classe `CompteBancaire` permettant :

- de déposer de l'argent,
- de retirer de l'argent (si le solde est suffisant),
- d'afficher le solde.

Correction

Fichier : `CompteBancaire.java`

```java
public class CompteBancaire {

    private double solde;

    public CompteBancaire(double soldeInitial) {
        this.solde = soldeInitial;
    }

    public void deposer(double montant) {
        if (montant > 0) {
            solde += montant;
        }
    }

    public void retirer(double montant) {
        if (montant > 0 && montant <= solde) {
            solde -= montant;
        } else {
            System.out.println("Retrait impossible.");
        }
    }

    public void afficherSolde() {
        System.out.println("Solde actuel : " + solde);
    }
}
```

Fichier : `TestCompte.java`

```java
public class TestCompte {
    public static void main(String[] args) {
        CompteBancaire c = new CompteBancaire(10000);
        c.deposer(5000);
        c.retirer(3000);
        c.afficherSolde();
    }
}
```

Résultat attendu

```
Solde actuel : 12000.0
```

Exercice 3 – Héritage : employés et salaire

Énoncé

Créer une classe `Employe` avec :

- nom,
- salaire de base.

Créer une classe `EmployePermanent` qui hérite de `Employe` et ajoute une prime.

Afficher le salaire total.

Correction

Fichier : `Employe.java`

```java
public class Employe {

    protected String nom;
    protected double salaireBase;

    public Employe(String nom, double salaireBase) {
        this.nom = nom;
        this.salaireBase = salaireBase;
    }

    public double calculerSalaire() {
        return salaireBase;
```

```
    }
}
```

Fichier : `EmployePermanent.java`

```
public class EmployePermanent extends Employe {

    private double prime;

    public EmployePermanent(String nom, double salaireBase, double prime) {
        super(nom, salaireBase);
        this.prime = prime;
    }

    @Override
    public double calculerSalaire() {
        return salaireBase + prime;
    }
}
```

Fichier : `TestEmploye.java`

```
public class TestEmploye {
    public static void main(String[] args) {
        Employe e = new EmployePermanent("Omar", 150000, 30000);
        System.out.println("Salaire : " + e.calculerSalaire());
    }
}
```

Résultat attendu

```
Salaire : 180000.0
```

Exercice 4 – Classe abstraite : formes géométriques

Énoncé

Créer une classe abstraite `Forme` avec une méthode abstraite `calculerAire()`.
Créer les classes `Rectangle` et `Cercle`.

Correction

Fichier : `Forme.java`

```
public abstract class Forme {
    public abstract double calculerAire();
}
```

Fichier : `Rectangle.java`

```
public class Rectangle extends Forme {

    private double longueur;
    private double largeur;

    public Rectangle(double longueur, double largeur) {
        this.longueur = longueur;
        this.largeur = largeur;
    }

    @Override
    public double calculerAire() {
        return longueur * largeur;
    }
}
```

Fichier : `Cercle.java`

```
public class Cercle extends Forme {

    private double rayon;

    public Cercle(double rayon) {
        this.rayon = rayon;
    }

    @Override
    public double calculerAire() {
        return Math.PI * rayon * rayon;
    }
}
```

Fichier : `TestForme.java`

```
public class TestForme {
    public static void main(String[] args) {
```

```
        Forme f1 = new Rectangle(5, 4);
        Forme f2 = new Cercle(3);

        System.out.println("Aire rectangle : " + f1.calculerAire());
        System.out.println("Aire cercle : " + f2.calculerAire());
    }
}
```

Exercice 5 – Interface : système de paiement

Énoncé

Créer une interface `Payable`.
Créer deux classes qui implémentent cette interface : `Facture` et `Employe`.

Correction

Fichier : `Payable.java`

```
public interface Payable {
    void payer(double montant);
}
```

Fichier : `Facture.java`

```
public class Facture implements Payable {

    @Override
    public void payer(double montant) {
        System.out.println("Facture réglée : " + montant);
    }
}
```

Fichier : `Employe.java`

```
public class Employe implements Payable {

    private String nom;

    public Employe(String nom) {
        this.nom = nom;
    }
```

```java
    @Override
    public void payer(double montant) {
        System.out.println(nom + " reçoit " + montant);
    }
}
```

Fichier : `TestPaiement.java`

```java
public class TestPaiement {
    public static void main(String[] args) {
        Payable p1 = new Employe("Ibrahima");
        Payable p2 = new Facture();

        p1.payer(100000);
        p2.payer(45000);
    }
}
```

Remarque

Cette série permet de couvrir :

- encapsulation,
- constructeurs,
- méthodes métier,
- héritage,
- polymorphisme,
- classes abstraites,
- interfaces.

Elle donne une **vision concrète et motivante** de la POO.

Compétences attendues

À l'issue de ce cours, il est attendu de savoir :

- créer des classes propres et cohérentes ;
- instancier et utiliser des objets ;
- appliquer l'encapsulation ;
- utiliser l'héritage et le polymorphisme ;
- comprendre et utiliser classes abstraites et interfaces.